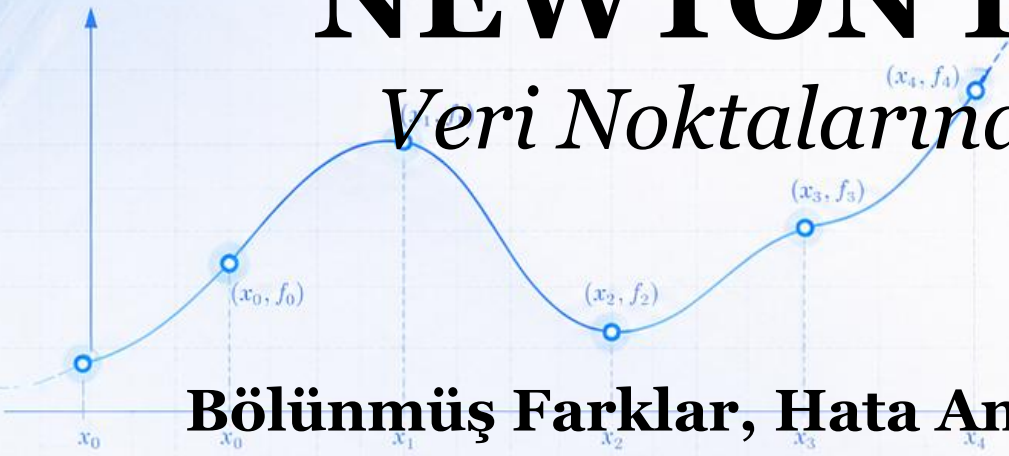
BIL2208 Nümerik Analiz 2025-2026 Bahar Dönemi

NEWTON İNTERPOLASYONU

Veri Noktalarından Mühendislik Tahminlerine



Bölünmüş Farklar, Hata Analizi ve MATLAB Tabanlı Mühendislik Uygulamaları

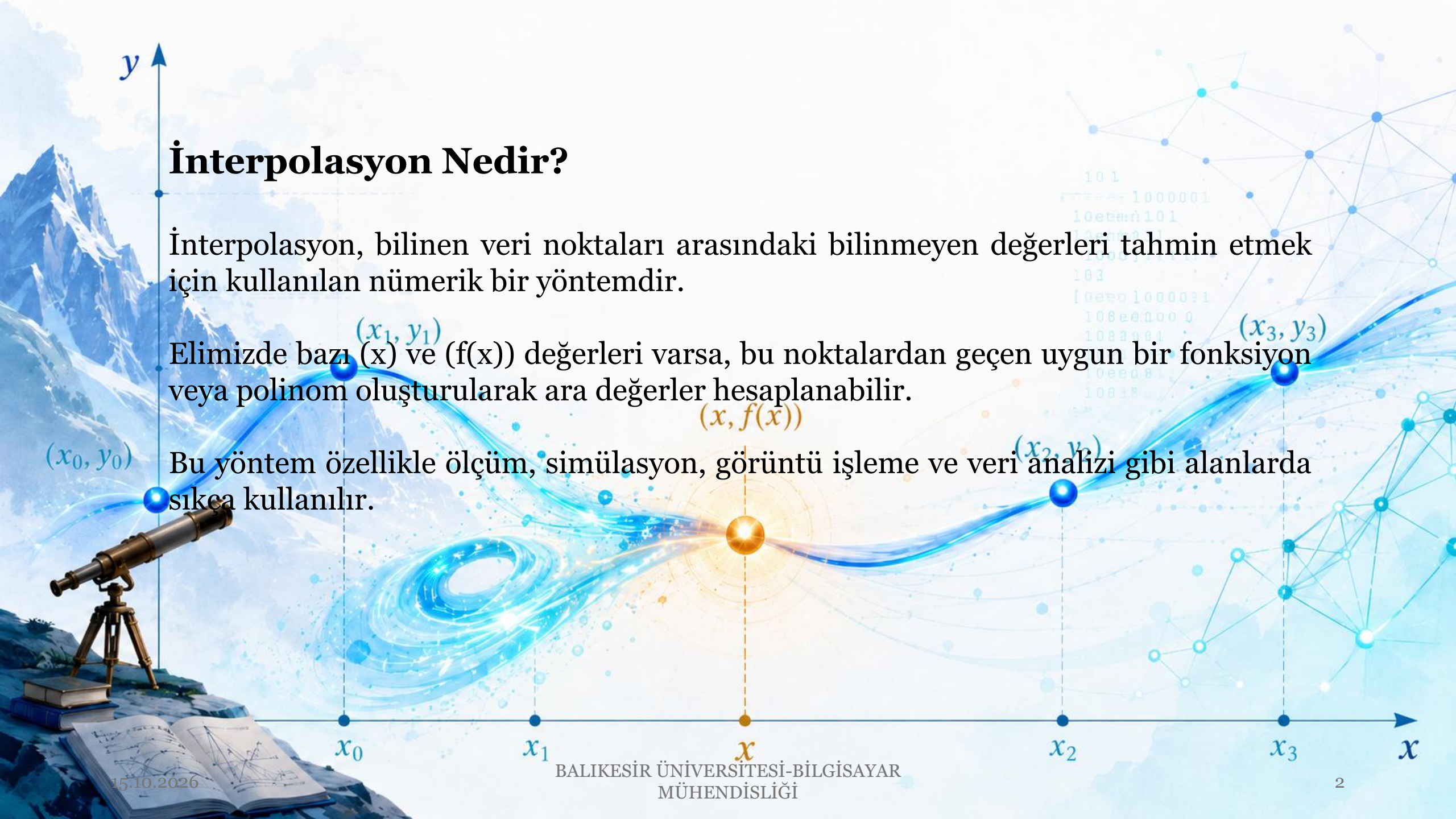
Hazırlayan: Büşra Akgün

İnterpolasyon Nedir?

İnterpolasyon, bilinen veri noktaları arasındaki bilinmeyen değerleri tahmin etmek için kullanılan nümerik bir yöntemdir.

Elimizde bazı (x) ve (f(x)) değerleri varsa, bu noktalardan geçen uygun bir fonksiyon veya polinom oluşturularak ara değerler hesaplanabilir.

Bu yöntem özellikle ölçüm, simülasyon, görüntü işleme ve veri analizi gibi alanlarda sıkça kullanılır.



Polinom İnterpolasyonu:

Polinom interpolasyonunda amaç, verilen veri noktalarından geçen bir polinom oluşturmaktır.

Genel bir (n). dereceden polinom şu şekilde yazılır:

$$f_n(x) = f(x_0) + (x - x_0)f[x_1, x_0] + (x - x_0)(x - x_1)f[x_2, x_1, x_0] + \cdots + (x - x_0)(x - x_1)\cdots(x - x_{n-1})f[x_n, x_{n-1}, \ldots, x_0]$$

(n+1) veri noktası için bu noktalardan geçen tek bir (n). dereceden interpolasyon polinomu vardır.

2 nokta → doğru, 3 nokta → parabol, 4 nokta → kübik eğri.

Newton İnterpolasyonunun Amacı:

Newton interpolasyonu, verilen veri noktalarından geçen polinomu bölünmüş farklar yardımıyla oluşturan bir yöntemdir.

Bu yöntemin önemli avantajı, polinomun adım adım kurulabilmesidir. Yeni bir veri noktası eklendiğinde tüm polinomu baştan oluşturmak yerine mevcut yapıya yeni bir terim eklenebilir.

Bu özellik Newton yöntemini bilgisayar uygulamaları için kullanışlı hale getirir.

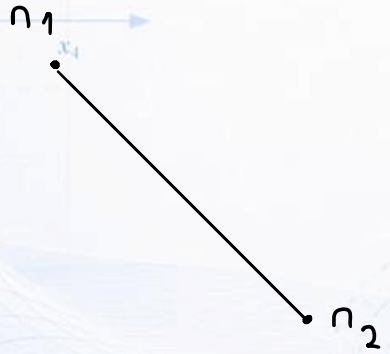
Lineer İnterpolasyon:

Lineer interpolasyon, iki veri noktası arasındaki değeri bu iki noktadan geçen doğru yardımıyla tahmin eder.

$$P_1(x) = f(x_0) + [(f(x_1) - f(x_0)) / (x_1 - x_0)](x - x_0)$$

Buradaki kesir, iki nokta arasındaki eğimi gösterir.

Aynı zamanda Newton yönteminde birinci bölünmüş fark olarak kullanılır.



Bölünmüş Fark Operatörü:

Bölünmüş fark, Newton interpolasyonunun temel yapı taşıdır.
Birinci dereceden bölünmüş fark:

$$f[x_0, x_1]=\{f(x_1) - f(x_0)\} / \{x_1-x_0\}$$

İkinci dereceden bölünmüş fark:

$$f[x_0, x_1, x_2]=\{f[x_1, x_2]-f[x_0, x_1]\} / \{x_2-x_0\}$$

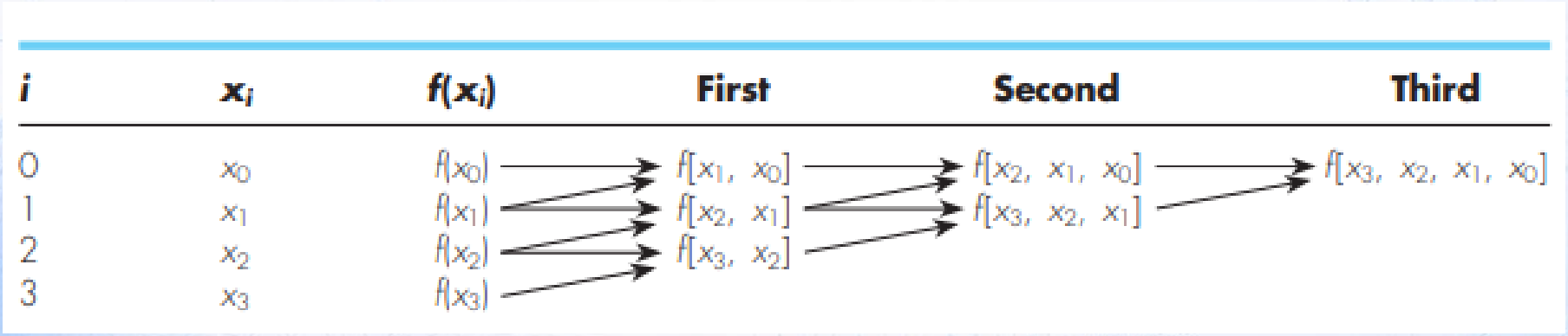
Bölünmüş farklar, polinomun katsayılarını belirlemek için kullanılır.

Bölünmüş Fark Tablosu:

Newton yönteminde katsayılar bölünmüş fark tablosu ile sistematik olarak hesaplanır.

Tabloda ilk sütunda (xi) değerleri, ikinci sütunda (f(xi)) değerleri bulunur. Sonraki sütunlarda ise birinci, ikinci ve daha yüksek dereceden bölünmüş farklar hesaplanır.

Newton polinomunun katsayıları tablonun üst satırından alınır.



1.Satırdaki sayılar katsayılardır.

Newton İnterpolasyon Polinomu:

Newton interpolasyon polinomunun genel formu:

$$f_n(x) = f(x_0) + (x - x_0)f[x_1, x_0] + (x - x_0)(x - x_1)f[x_2, x_1, x_0] + \cdots + (x - x_0)(x - x_1)\cdots(x - x_{n-1})f[x_n, x_{n-1}, \ldots, x_0]$$

Bu yapı sayesinde polinom derece derece genişletilir.

İlk terim başlangıç değerini, sonraki terimler ise eğim ve eğrilik bilgisini temsil eder.

Kuadratik Newton Polinomu:

Üç veri noktası kullanıldığında ikinci dereceden Newton polinomu elde edilir.

$$P_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$



Burada:

$b_0 = f(x_0)$

$b_1 = f[x_0, x_1]$

$b_2 = f[x_0, x_1, x_2]$

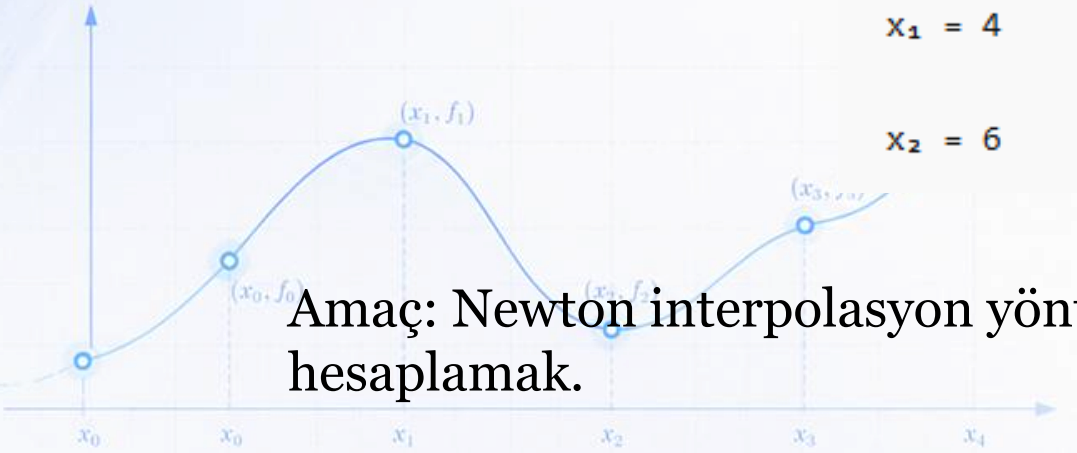
(b2) terimi polinoma eğrilik kazandırır.

Doğal logaritma fonksiyonuna ait üç veri noktası verilsin:

$x_0 = 1 \qquad f(x_0) = \ln(1) = 0$

$x_1 = 4 \qquad f(x_1) = \ln(4) = 1.386294$

$x_2 = 6 \qquad f(x_2) = \ln(6) = 1.791759$



Amaç: Newton interpolasyon yöntemiyle (ln 2) değerini yaklaşık olarak hesaplamak.

Örnek Problem Katsayıları

İlk katsayı:

b0= f(x0)= 0

Birinci bölünmüş fark:

b1= {1.386294-0}/{4-1}=0.4620981

İkinci bölünmüş fark:

$$b_2 = \frac{\frac{1.791759-1.386294}{6-4} - 0.4620981}{6-1} = -0.0518731$$

Newton Polinomunun Kurulması:

Bulunan katsayılar polinomda yerine yazılır:

$$P2(x)= 0 + 0.4620981(x-1) - 0.0518731(x-1)(x-4)$$

Bu polinom, verilen üç noktadan geçen ikinci dereceden Newton interpolasyon polinomudur.

(x=2) için:

$$P2(2)= 0.5658444$$

Gerçek değer:

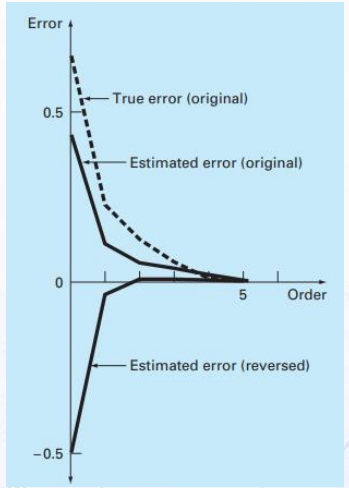
$$\ln 2= 0.6931472$$

Hata Analizi:

İnterpolasyon polinomu gerçek fonksiyona yaklaşık bir model oluşturur. Bu nedenle gerçek değer ile tahmin edilen değer arasında hata oluşabilir.
Newton interpolasyonu için hata terimi genel olarak şu şekilde ifade edilir:

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n)$$

Hata, seçilen noktaların konumuna, polinom derecesine ve fonksiyonun türev davranışına bağlıdır.



Lineer ve Kuadratik Tahmin Karşılaştırması:

Lineer interpolasyon iki nokta kullanır ve düz çizgi ile tahmin yapar. Kuadratik interpolasyon üç nokta kullanır ve eğriliği de modele dahil eder. Bu nedenle eğri fonksiyonlarda kuadratik interpolasyon çoğu zaman lineer interpolasyona göre daha iyi sonuç verebilir.

Örnek problemde:
Lineer tahmin:

(0.4620981)

Kuadratik tahmin:

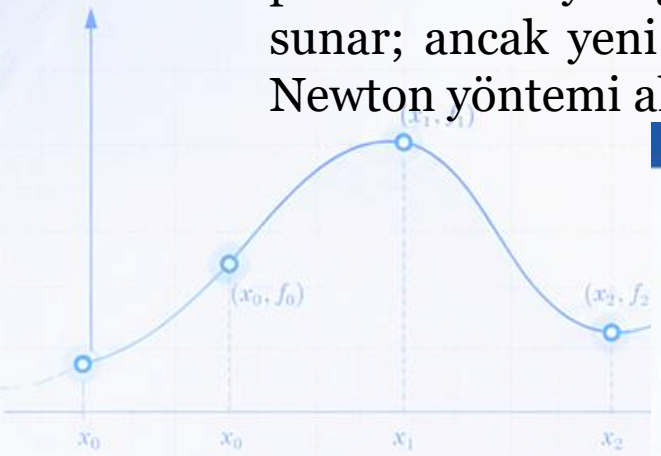
(0.5658444)

Gerçek değer:

(0.6931472)

Newton ve Lagrange Karşılaştırması:

Newton ve Lagrange interpolasyonu aynı interpolasyon polinomunu farklı biçimlerde ifade eder.Newton yöntemi bölünmüş fark tablosu kullanır ve yeni veri noktası eklendiğinde polinom kolayca genişletilebilir. Lagrange yöntemi teorik olarak daha doğrudan bir form sunar; ancak yeni veri eklendiğinde polinomun büyük kısmı yeniden hesaplanır.Bu nedenle Newton yöntemi algoritmik uygulamalar için daha pratiktir.



NEWTON INTERPOLATION

1. Divided Differences Table

i	x_i	$f[x_i]$	$f[x_i,x_{i+1}]$	$f[x_i,x_{i+1},x_{i+2}]$	...
0	x_0	f_0	$f_{0,1}$	$f_{0,1,2}$	...
1	x_1	f_1	$f_{1,2}$	$f_{1,2,3}$	
2	x_2	f_2	$f_{2,3}$		
3	x_3	f_3			
⋮	⋮	⋮			

$$f[x_i,x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

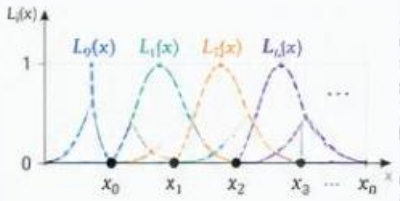
2. Incremental Polynomial Construction

$$\begin{aligned} P_n(x) &= f[x_0] \\ &+ f[x_0,x_1](x - x_0) \\ &+ f[x_0,x_1,x_2](x - x_0)(x - x_1) \\ &+ \cdots \\ &+ f[x_0,\ldots,x_n](x - x_0)\cdots(x - x_{n-1}) \end{aligned}$$

LAGRANGE INTERPOLATION

1. Basis Polynomials

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$



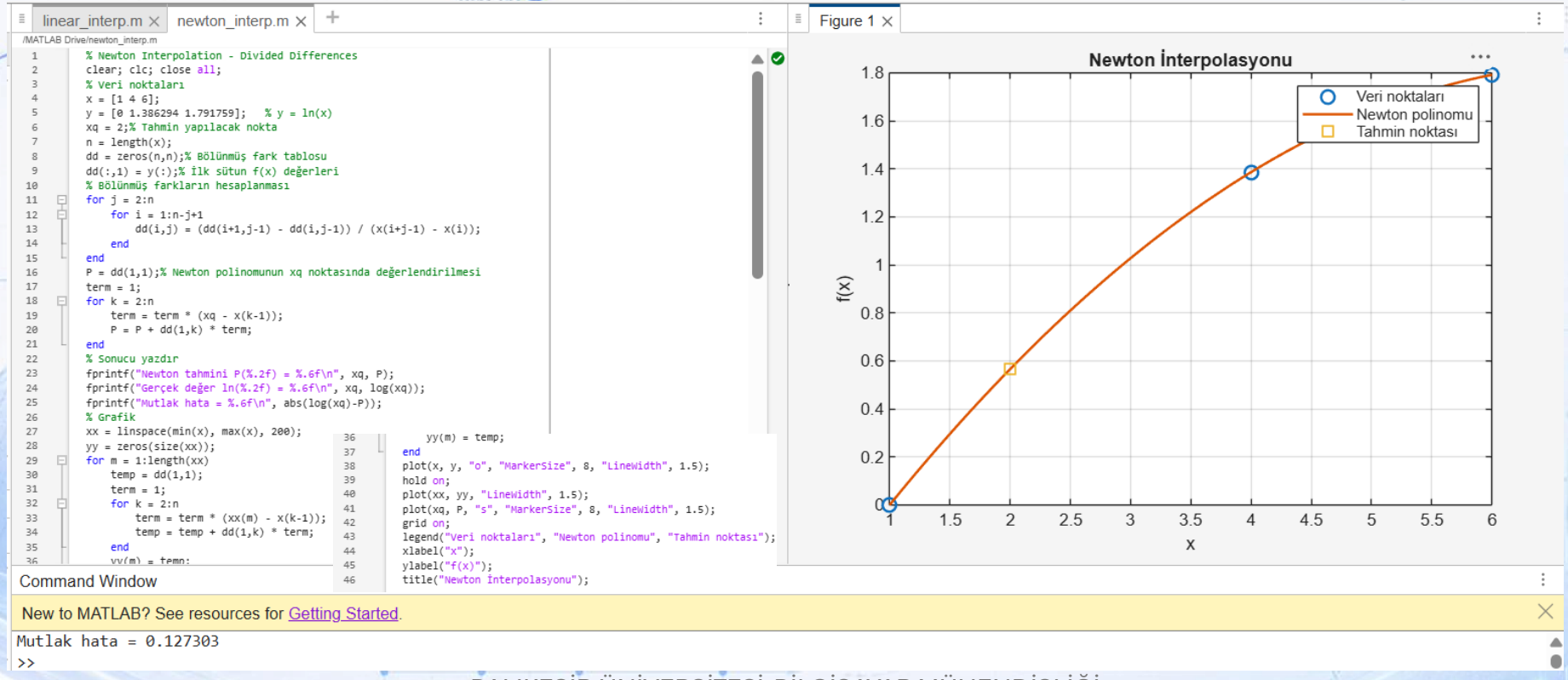
2. Direct Polynomial Construction

$$P_n(x) = \sum_{i=0}^{n} f(x_i)\, L_i(x)$$

Görsel: https://app.leonardo.ai/generate?model=auto-preset

MATLAB ile Newton İnterpolasyonu:

Newton interpolasyonu MATLAB ortamında kolayca uygulanabilir.
Algoritmanın temel adımları: (x) ve (f(x)) değerlerini tanımlamak,
Bölünmüş fark tablosunu oluşturmak Katsayıları tablodan almak,
Newton polinomunu istenen noktada değerlendirmek,
Sonucu grafik üzerinde göstermek,
Bu yapı, yöntemin bilgisayar mühendisliği açısından algoritmik bir problem olarak ele alınmasını sağlar.



Bilgisayar Mühendisliğinde Uygulamalar:

Newton interpolasyonu ve genel olarak interpolasyon yöntemleri bilgisayar mühendisliğinde birçok alanda kullanılır.

Başlıca uygulamalar: Sensör verilerinde eksik değer tahmini, görüntü yeniden boyutlandırma işlemleri, simülasyon verilerinde ara değer hesaplama, sayısal modelleme, veri analizi ve tahmin problemleri, grafik ve animasyonlarda geçişlerin yumuşatılması,...

Bu nedenle interpolasyon yalnızca teorik bir matematik konusu değil, aynı zamanda hesaplamalı mühendislik aracıdır.

Uygulama 1: Eksik Veri Tamamlama

linear_interp.m

/MATLAB Drive/linear_interp.m

```
%Eksik veri oluşturup sonra interpolasyonla doldurma

x = [-4*pi: 0.1 :0, 0.1: 0.2: 4*pi];%[-4*pi: 0.1 :0 ]: -4π'den 0'a kadar 0.1
% aralıklarla değer üretir.
%[ 0.1: 0.2: 4*pi ] Bu kısım 0.1'den 4π'ye kadar 0.2 aralıklarla değer üretir

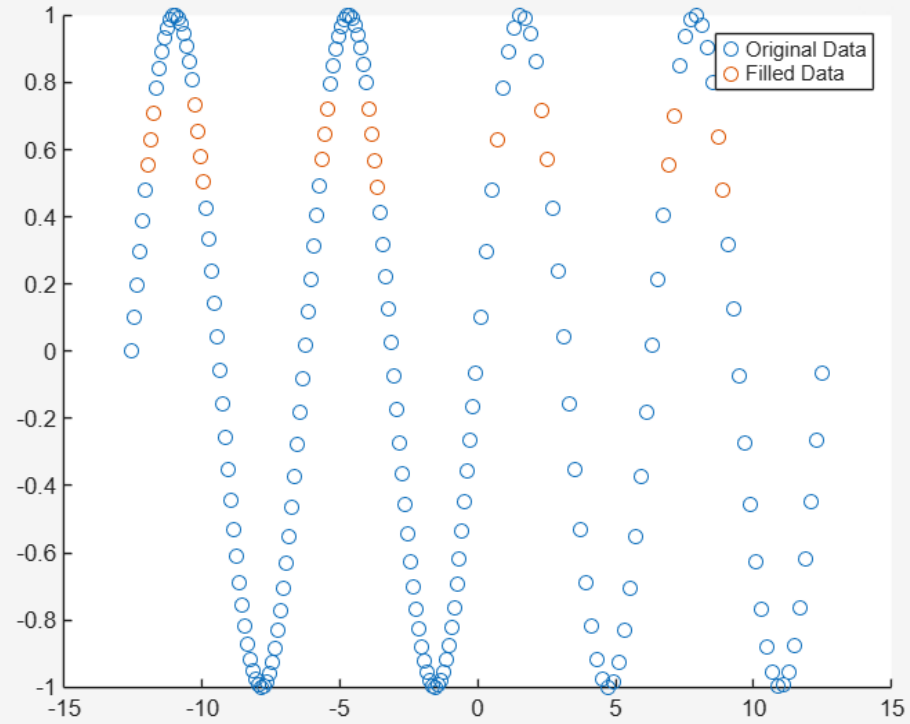
A = sin(x);% Her x değeri için sinüs değeri hesaplanır.
%A, ölçüm verisi gibi düşünülebilir. Mesela x: Zaman A: Sensörden gelen
%sinyal değeri. Yani elimizde bir sinyal  var

A(A < 0.75 & A > 0.5)= NaN;%Bu satır bazı veri nooktalarını eksik veri haline getiriyor.
%NaN: Not a Nımber anlamına gelir yani burada geçerli bir sayı yok.
% 0.5 ve 0.75 ise o değeri sil veNaN olarak belirle.
%***Sensör bazı aralıklarda veri gönderemedi veya ölçüm alınamadı.

[F,TF] = fillmissing(A,"linear",SamplePoints=x);%fillmissing, Matlabın eksik
% veri doldurmak için kullanılan fonksiyonudur.
% A:Eksik değerler içeren veri dizisi.
% "linear":Eksik değerler doğrusal interpolasyonla doldurulacak.
% Samplepoints=x: verilerin hangi x noktalarına karşılık geldiğini söyler.
% F: Eksiklerin tamamlanmış oldupu yeni dizi. TF: Hangi verilerin doldurulduğunu
% gösteren mantıksal dizi. TF içinde true olan yerler: Matlab'ın doldurduğu
% false olan yerler: Zaten var olan noktalar.

scatter(x,A)% Bu satır orijinal veriyi nokta grafik olarak çizer. Yatay eksen: x
% Dikey eksen:A
hold on%Önceki grafiği silme, yeni çizii aynı grafik üzerine ekle.
scatter(x(TF),F(TF))%Bu satır sadece Matlab'ın doldurduğu noktaları çizer.
% x(TF): Eksik veri olan x konumlarını seçer.
% F(TF): Bu konumlarda interpolasyonla hesaplanan yeni değerleri seçer
legend("Original Data","Filled Data")
```

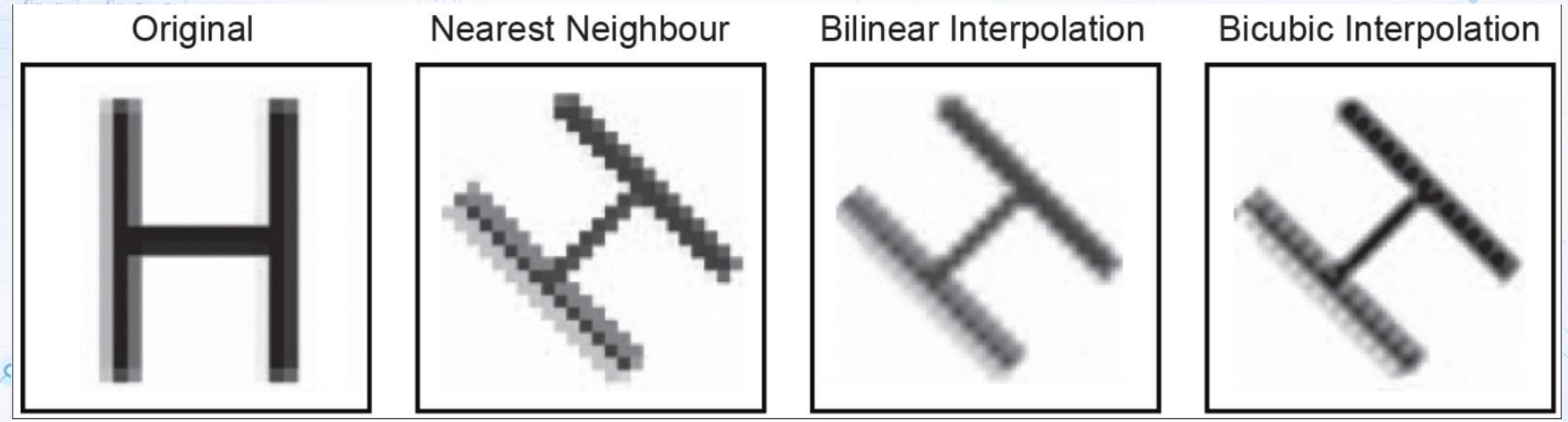
Figure 1



Eksik veya bozulmuş veri noktaları, interpolasyon yardımıyla komşu değerlerden tahmin edilebilir. MATLAB'da fillmissing fonksiyonu, doğrusal interpolasyon gibi yöntemlerle NaN değerlerini doldurmak için kullanılabilir. Bu yaklaşım sensör verileri, zaman serileri ve ölçüm sistemlerinde sıkça kullanılır.

Fikir: https://www.mathworks.com/help/matlab/ref/fillmissing.html

Uygulama 2: Görüntü Yeniden Boyutlandırma Örnek 1



Görüntü büyütme veya döndürme işlemlerinde yeni piksellerin değerleri doğrudan bilinmez. İnterpolasyon, komşu piksellerden yararlanarak bu değerleri tahmin eder. Görselde en yakın komşu, Bilineer ve Bikübik yöntemlerinin aynı görüntü üzerinde farklı sonuçlar ürettiği görülmektedir.

Görsel: https://ltb.itc.utwente.nl/498/concept/81586

Uygulama 2: Görüntü Yeniden Boyutlandırma Örnek 2:

Farklı interpolasyon yöntemleri aynı görüntü üzerinde farklı keskinlik ve yumuşaklık sonuçları üretir. Görsel: OpenCV ile farklı interpolasyon yöntemlerinin görüntü büyütme üzerindeki etkisi.



Örnek 2 Programı:

gorsel.py > ...

```python
import cv2#Opencv kütüphanesi programa dahile ediliyor. Görüntüyü okumak ve yeniden
# boyutlandırmak içiin kullanılır
import matplotlib.pyplot as plt #Görselleri ekranda ekranda tutmak için kullanılır.
# Kısaca plt olarak çağırılabilir
# Resmi oku
img = cv2.imread("kedi.png")
if img is None:#Dosya bulunamazsa kısaca uyarı verir. Boş görüntü ile işlem yapamayız.
    print("Resim okunamadı. Dosya adını ve klasörü kontrol et.")
    exit()
# OpenCV BGR sırasıyla (Blue-Green-Red) okur, matplotlib RGB(Red-Green-Blue) ister
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)#Renkler doğru görünsün diye dönüşüm yapar.
# Fark daha net görünsün diye 6 kat büyütüyoruz
scale = 6
nearest = cv2.resize(img_rgb, None, fx=scale, fy=scale,
                     interpolation=cv2.INTER_NEAREST)#görüntü en yakın komşu yöntemiyle büyütülür.
#En yakın piksel değeri kopyalanır. Sonuç daha bloklu ve pikselli görünür.

bilinear = cv2.resize(img_rgb, None, fx=scale, fy=scale,
                      interpolation=cv2.INTER_LINEAR)#Görüntü Bilineer interpolasyon ile bütyütülür.
#Komşu piksellerden ara değer hesaplar. Sonuç yumuşak ama biraz bulanık olur.

bicubic = cv2.resize(img_rgb, None, fx=scale, fy=scale,
                     interpolation=cv2.INTER_CUBIC)#Görüntü bikübik interpolasyon ile büyütülür.
#Daha fazla komşu pikseli hesaba katar. Genellikle daha yumuşak ve daha dengeli sonuç verir.

# 4 resmi aynı pencerede gösterecek.
images = [img_rgb, nearest, bilinear, bicubic]
titles = ["Original", "Nearest Neighbor", "Bilinear", "Bicubic"]# Başlıklar listeleniyor.

plt.figure(figsize=(12, 6))#Açılacak grafik penceresinin boyutunu ayarlar.

for i in range(4):
    plt.subplot(2, 2, i + 1)#2x2 düzeninde gösterir.
    plt.imshow(images[i])
    plt.title(titles[i])#Her birine başlık ekler.
    plt.axis("off")#pencere kapatılır.

plt.tight_layout()#Görsellerin düzgün yerleşmesini sağlar
plt.show()#Pencereyi ekrana getirir.
```

Uygulama 3: Uydu Verilerinde Eksik Gözlem Tamamlama



(h) Input (full scene)

(i) VPint2 (full scene)

Uydu görüntülerinde bulut örtüsü veya sensör kaynaklı eksik bölgeler oluşabilir. İnterpolasyon ve benzeri tamamlama yöntemleri, komşu uzamsal veya zamansal bilgilerden yararlanarak bu boşlukların yaklaşık olarak doldurulmasına yardımcı olur. Bu yaklaşım tarım, çevresel izleme ve iklim analizinde önemlidir.

Görsel: https://www.sciencedirect.com/science/article/pii/S0924271624002995

Sonuç:

Newton interpolasyonu, veri noktalarından geçen bir polinom oluşturmak için kullanılan etkili bir nümerik yöntemdir.

Bölünmüş fark tablosu sayesinde katsayılar düzenli biçimde hesaplanır. Yöntem, yeni veri noktalarının eklenmesine uygun olduğu için bilgisayar algoritmaları açısından avantaj sağlar.

Bu çalışmada Newton interpolasyonunun matematiksel temeli, hata analizi, örnek çözümü, Lagrange yöntemiyle karşılaştırması ve bilgisayar mühendisliğindeki uygulamaları incelenmiştir.

Kaynakça ve Yapay Zeka Kullanım Beyanı:

Bu sunumun hazırlanma sürecinde üretken yapay zekâ araçlarından destek alınmıştır. Yapay zekâ; konu anlatımının sadeleştirilmesi, sunum taslağının düzenlenmesi, görsel tasarım fikirleri ve metinlerin akademik ifade açısından geliştirilmesi amacıyla kullanılmıştır.
Matematiksel içerikler, formüller ve örnek çözümler kaynaklar doğrultusunda kontrol edilmiştir. Nihai düzenleme ve içerik seçimi tarafımdan yapılmıştır.

Görsel: https://www.sciencedirect.com/science/article/pii/S0924271624002995

Fikir: https://www.mathworks.com/help/matlab/ref/fillmissing.html

Görsel: https://ltb.itc.utwente.nl/498/concept/81586

Görsel: https://app.leonardo.ai/generate?model=auto-preset

2. Slayt arka planı ve diğer arka plan Chatgpt ile üretilmiştir.

Konu buradan öğrenildi ve formüller buradan alındı. https://www.mbit.edu.in/wp-content/uploads/2020/05/Numerical_methods_for_engineers_for_engi.pdf

Kodlara ulaşmak için tıklayın: https://github.com/busraakgun24/numerical-analysis-and-matlab-projects